

LAKEHOUSE ARCHITECTURE FOR E-COMMERCE TRANSACTIONS

An e-commerce company wants a production-grade **Lakehouse architecture** on **AWS**. The system will ingest raw transactional data stored in Amazon S3, clean and deduplicate it using Delta Lake, and expose it for downstream analytics through Amazon Athena. Orchestrating the entire process using AWS Step Functions and automating deployment with CI/CD on GitHub Actions. The business expects high data reliability, schema enforcement, and consistent data freshness for analytical workloads.

The business expects high data reliability, schema enforcement, and consistent freshness of data for analytical workloads.

SCHEMA

PRODUCT TABLE

Column Name	Type	Description
product_id	INT	Unique Identifier for each product.
department_id	INT	Foreign Key to the department this product belongs to.
department	String	Name of the department

Product name	String	Descriptive name of the product.
--------------	--------	----------------------------------

ORDER TABLE

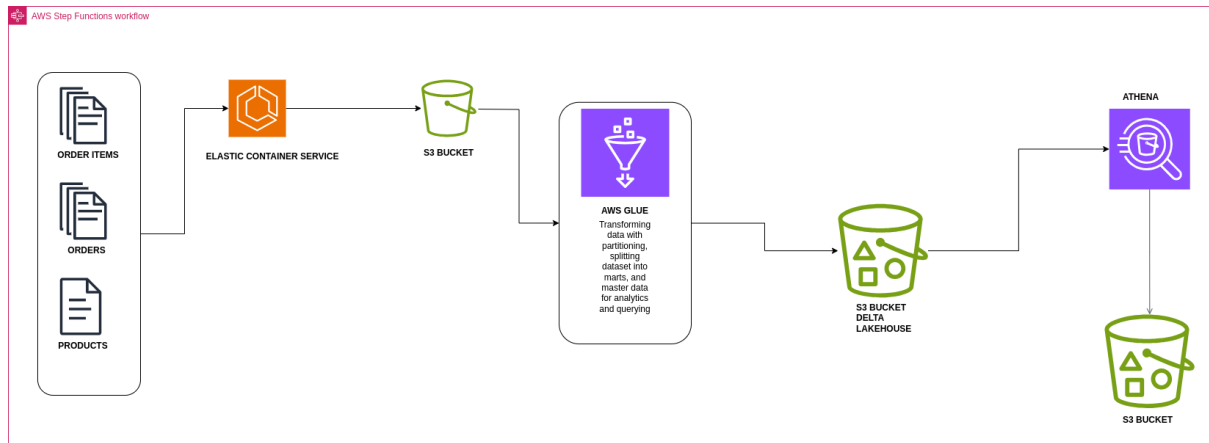
Column Name	Data Type	Description
order_num	Integer	Sequence number of orders per user
order_id	Integer	Unique identifier for each order
user_id	Integer	Unique identifier of the user who placed the order
order_timestamp	Timestamp	Date and time when the order was placed
total_amount	Float	Total cost of the order
date	Date	Date part of the order (used for easier filtering)

ORDERS ITEMS

Column Name	Type	Description
-------------	------	-------------

id	Integer	Unique identifier for each item record
order_id	Integer	Foreign key linking to the Orders table
user_id	Integer	ID of the user who added the item to the cart
days_since_prior_order	Integer	Days since this user's previous order; null for first orders
product_id	Integer	ID of the product ordered
add_to_cart_order	Integer	Order in which the item was added to the cart (e.g., 1st, 2nd item)
reordered	Boolean	Indicates if the product was previously ordered by the user
order_timestamp	Timestamp	Date and time when the item was ordered
date	Date	Date part of the order timestamp for aggregation

FLOW DIAGRAM



ORCHESTRATION

Overview

This orchestration pipeline is designed to automate the end-to-end data processing lifecycle for an e-commerce data lakehouse on AWS. It leverages multiple AWS services to load raw data into an S3 bucket, process it using AWS Glue, and run analytical queries with Amazon Athena. At each critical point in the pipeline, monitoring and notification mechanisms are integrated to handle failures and alert relevant stakeholders.

Orchestration Flow Summary

Data Ingestion:

- The pipeline begins by invoking a containerized ECS Fargate task that loads raw transactional or event-based data into a designated S3 bucket. This data serves as the input for the lakehouse architecture.

Data Availability Check:

- Once the ingestion task is complete, the pipeline verifies whether the data was successfully written to the S3 bucket by listing objects under a specified prefix.

- If no data is found, the pipeline sends a notification and terminates, preventing downstream processes from executing unnecessarily.

Data Transformation:

- If data is present, an AWS Glue job is triggered to clean, transform, and organize the raw data into a structured format that is lakehouse-ready.
- This transformation might include parsing, enrichment, partitioning, or format conversion (e.g., from CSV to Parquet).
-

Metadata Cataloging:

- Following successful transformation, an AWS Glue Crawler is started to scan the newly transformed data and create metadata entries (tables) in the AWS Glue Data Catalog.
- These entries enable the use of Athena for querying the dataset.

Metadata Validation:

- After giving the crawler some time to complete, the pipeline validates the presence of tables in the Glue Data Catalog to ensure metadata registration was successful.
- Failure to detect tables results in a termination with appropriate alerts.

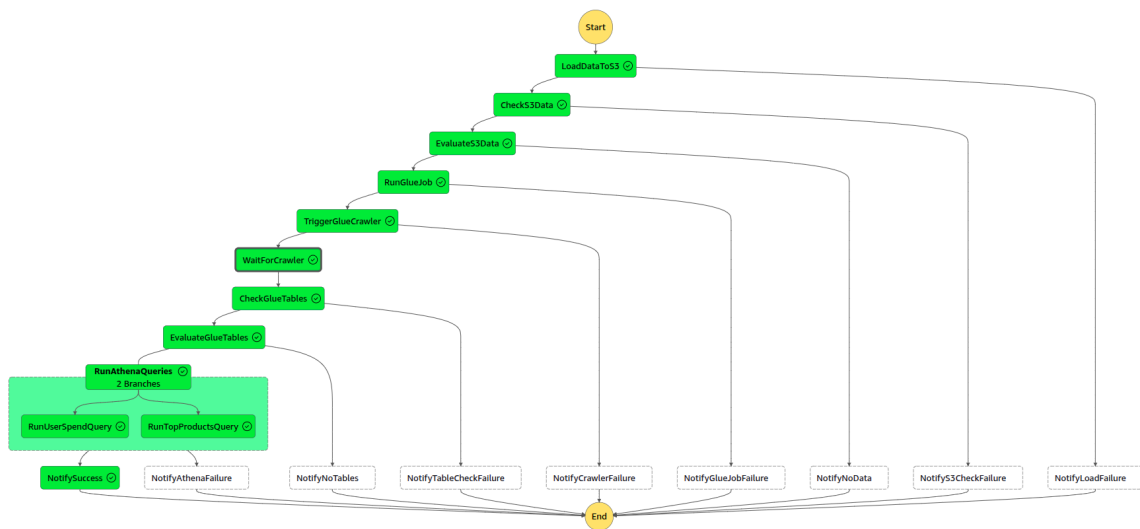
Data Analytics:

- If metadata is successfully cataloged, the pipeline proceeds to execute a set of parallel Athena queries for analytical reporting.

- Examples include user spend tracking and identifying top-performing products.

Completion and Notifications:

- Upon successful completion of all tasks, the pipeline sends a success notification.
- At every critical step, failure handlers are in place to notify stakeholders via SNS with detailed error messages to aid in debugging.



Key Orchestration Concepts

Event-Driven Execution: The pipeline is built using AWS Step Functions, which allow sequential and parallel state transitions based on the success or failure of each step.

Conditional Branching: Logic branches ensure that the pipeline adapts based on real-time outcomes, e.g., halting if no data is found or continuing if tables are successfully cataloged.

Service Coordination: It orchestrates various AWS services such as ECS, S3, Glue, and Athena in a coordinated flow that ensures data reliability, schema consistency, and automation of analytics.

Fault Tolerance: Built-in catch and notification mechanisms ensure that any failure is captured, logged, and communicated, thus improving pipeline reliability and debuggability.

Modular & Extensible: The structure supports plug-and-play modifications—new queries, datasets, or transformations can be added without altering the fundamental orchestration logic.

Benefits

Automation: Fully automates the ingestion-to-analysis process.

Scalability: Leverages serverless and managed services that scale with demand.

Reliability: Monitors each stage and handles errors gracefully.

Observability: Offers transparency into pipeline status and results.

AUTOMATION

Overview

This repository uses GitHub Actions to automate the testing and deployment of AWS Glue ETL scripts. The workflow is triggered when code changes are pushed or pull requests are created for specific directories. The primary goal is to streamline data pipeline updates, ensure code quality through testing, and automate deployment to AWS Glue and subsequent orchestration.

Automation Workflow

Trigger Conditions

The workflow is triggered automatically on:

- Push events to the main branch affecting the Glue script directory.
- pull_request events targeting the main branch and modifying the same directory.

Workflow Jobs

1. Testing Job (test)

Objective: Ensure code reliability before deployment.

Actions:

- Check out the repository.
- Sets up Python and Java environments (required for PySpark).
- Installs dependencies from requirements.txt.
- Runs pytest integration tests inside the Glue script folder.

2. Deployment Job (deploy)

Triggered only after tests pass and on push events to main.

Actions:

- Check out the codebase.
- Configures AWS credentials securely via GitHub Secrets.
- Uploads the updated Glue script(s) to an S3 location used by AWS Glue.

- Updates the corresponding AWS Glue Job using the AWS CLI.
- Automatically triggers the downstream processing pipeline via Step Functions.

Benefits of This Automation

Consistency: Ensures the latest validated scripts are always deployed.

Speed: Eliminates manual AWS Console interactions.

Reliability: Integration tests catch issues early in the CI/CD pipeline.

Security: Uses GitHub Secrets to manage AWS credentials.

Scalability: Can be extended to multiple Glue jobs or pipelines with minimal changes.